

InterviewCoach AI Architecture

InterviewCoach AI — Architecture Document

Table of contents

1. Executive summary
2. High Level Design (HLD)
 - 2.1 Purpose and scope
 - 2.2 System context
 - 2.3 Logical architecture
 - 2.4 Physical deployment (AWS)
 - 2.5 Component summary
 - 2.6 Key user journeys
 - 2.7 External integrations
 - 2.8 Non-functional requirements
 - 2.9 CI/CD pipeline
3. Low Level Design (LLD)
 - 3.1 Repository structure
 - 3.2 Frontend
 - 3.3 Backend processes
 - 3.4 Configuration
 - 3.5 Backend modules (backend/common/)
 - 3.6 Interview engine (backend/INTERVIEW/)
 - 3.7 API endpoints (grouped)
 - 3.8 Database schema
 - 3.9 Network design
 - 3.10 Infrastructure inventory
 - 3.11 Server directory layout
 - 3.12 Deploy sequence
 - 3.13 Security controls
 - 3.14 Observability
 - 3.15 Capacity and tuning
 - 3.16 Local development
4. Appendix
 - 4.1 Diagram files (professional AWS icons)
 - 4.2 Regenerate Word/PDF exports
 - 4.3 Source references

InterviewCoach AI — Architecture Document

Version: 1.0
Last updated: 2026-06-23
Production URL: <https://ugaanlabs.ai>
AWS Region: ap-south-1 (Mumbai)
Infrastructure layout: Plan B (split Frontend | API | AI | RDS)

Related docs: [DEPLOY.md](#) · [DEVOPS_HANDOFF.md](#) · [PRODUCTION_CHECKLIST.md](#)

Table of contents

- 1. [Executive summary](#)
- 2. [High Level Design \(HLD\)](#)
- 3. [Low Level Design \(LLD\)](#)
- 4. [Appendix](#)

1. Executive summary

InterviewCoach is an AI-powered mock interview platform. Users upload resumes and job descriptions, configure interview questions, conduct live voice/text mock interviews, and receive structured feedback and performance analytics.

The production system runs on AWS with a **four-tier split**:

Tier	Host	Role
Presentation	Frontend EC2 (t3.small)	React SPA + nginx reverse proxy
Application	API EC2 (c6i.large)	Flask/gunicorn business logic
AI	AI EC2 (c6i.2xlarge)	Ollama LLM + Whisper transcription sidecar
Data	RDS PostgreSQL (db.t3.medium)	Persistent storage

EC2 and RDS run **Mon–Fri 10:00–19:30 IST** only (weekends off). EventBridge schedules in Asia/Kolkata.

2. High Level Design (HLD)

2.1 Purpose and scope

In scope

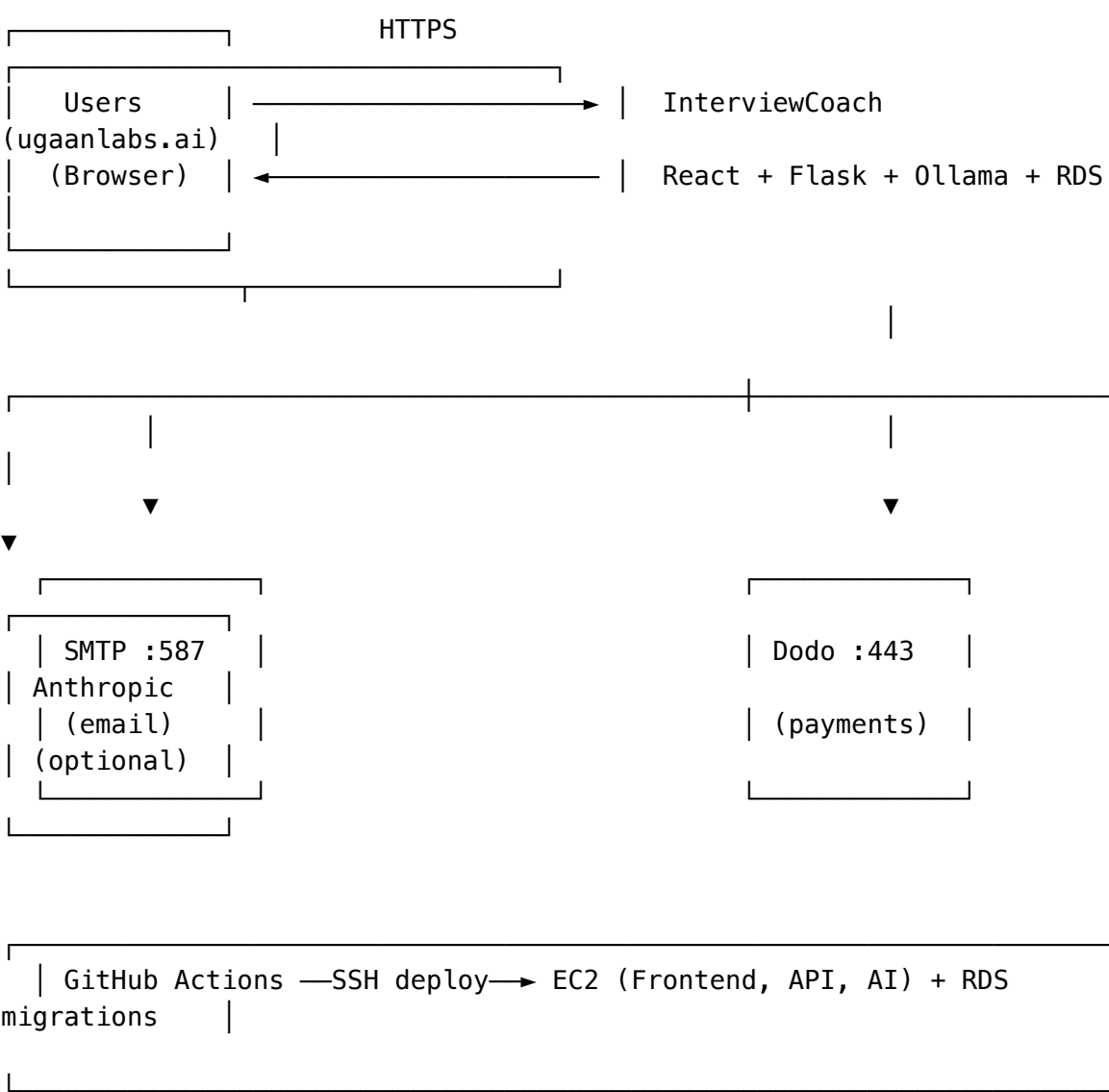
- User registration, login, email verification

- Resume and job description upload/parsing
- AI mock interviews (voice + text)
- Interview feedback and performance trends
- Payments (Dodo Payments gateway)
- Admin log viewer for operations
- CI/CD via GitHub Actions

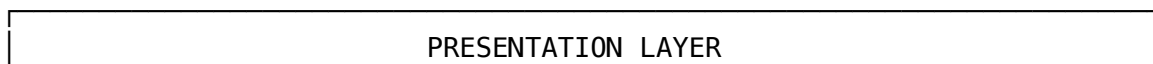
Out of scope (current)

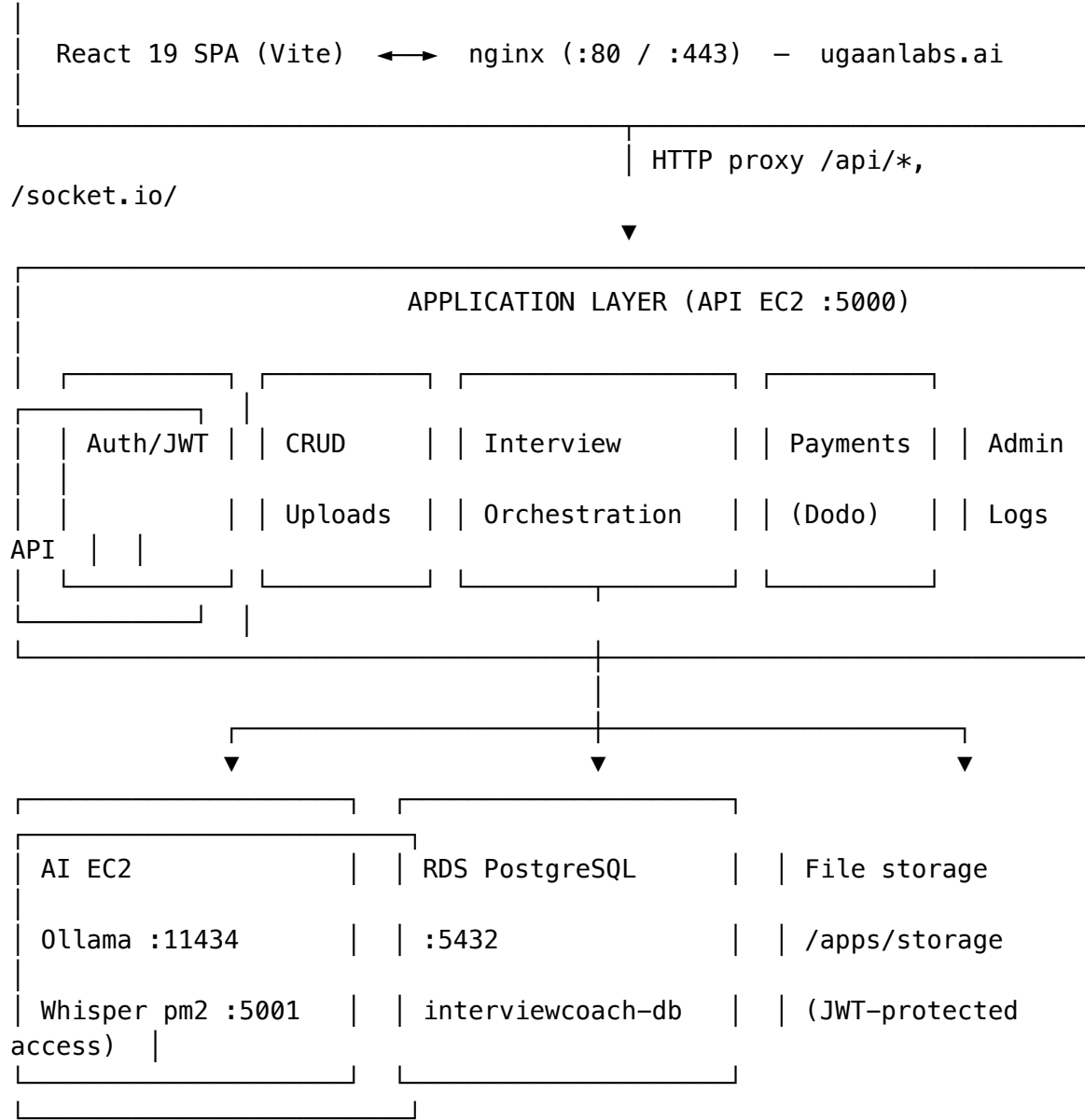
- Multi-region high availability
- Redis / multi-node API cluster
- CloudFront CDN
- 100+ parallel live AI interviews on current budget

2.2 System context

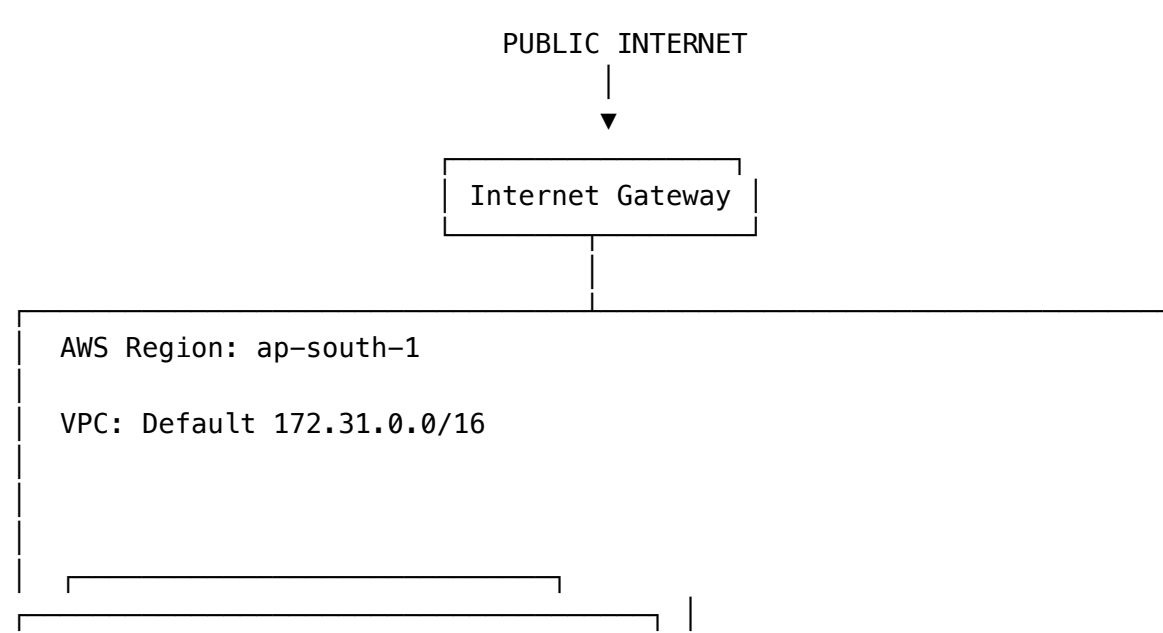


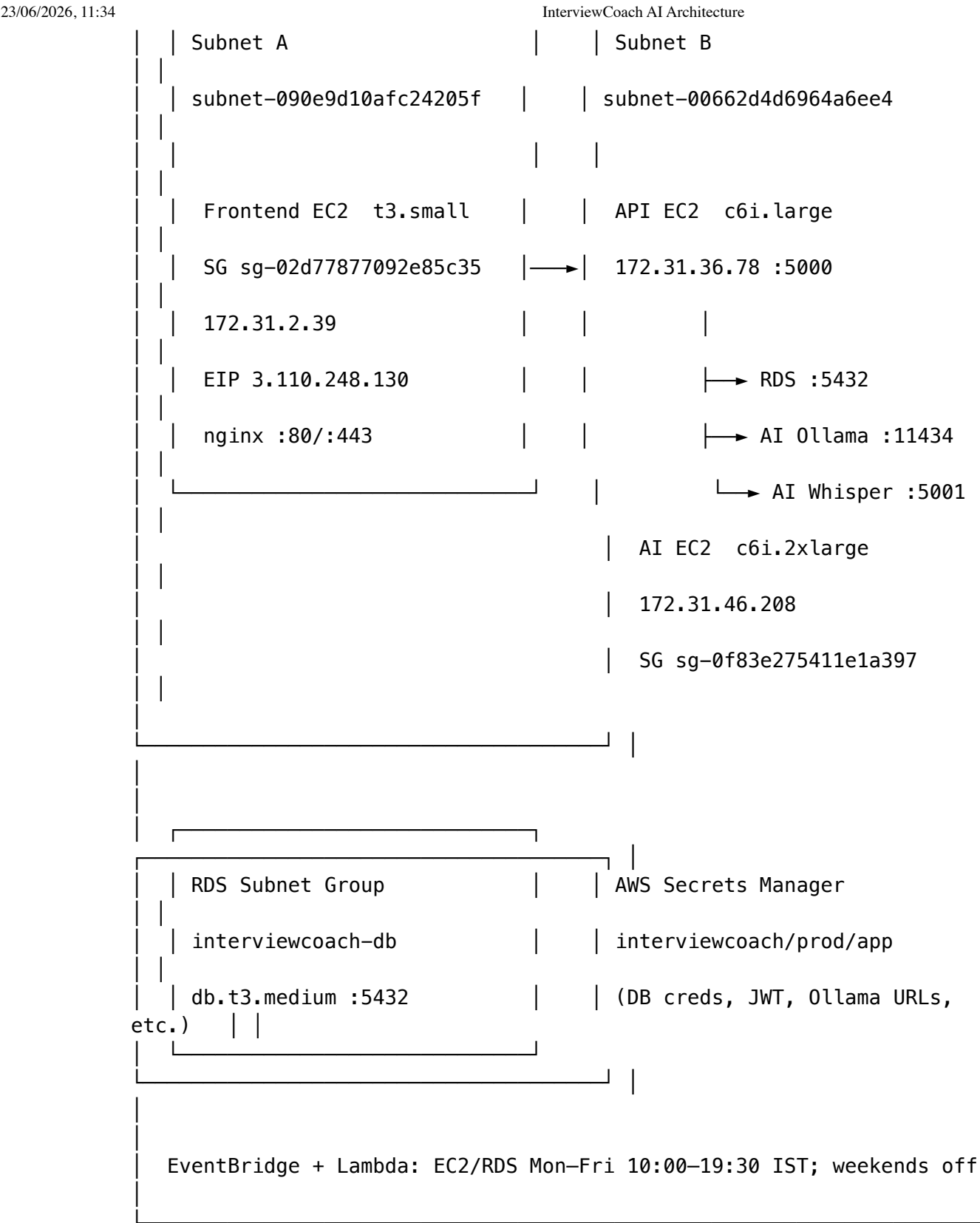
2.3 Logical architecture





2.4 Physical deployment (AWS)





2.5 Component summary

Component	Technology	Host	Role
Frontend UI	React 19, Vite, Tailwind 4	Frontend EC2	SPA, interview UI, dashboard

Component	Technology	Host	Role
Edge proxy	nginx	Frontend EC2	TLS termination, static files, API proxy
API server	Flask 3, gunicorn, Socket.IO	API EC2	Business logic, auth, orchestration
LLM	Ollama (llama3.2:3b)	AI EC2	Interview replies, optional question gen
STT	faster-whisper (pm2 sidecar)	AI EC2	Voice-to-text (VPC-internal only)
Database	PostgreSQL	RDS	Persistent relational data
Secrets	AWS Secrets Manager	Regional	Production configuration
CI/CD	GitHub Actions	External	Build, test, SSH deploy

2.6 Key user journeys

Interview flow

1. User creates interview via POST /api/interviews.
2. API persists interview and questions in RDS.
3. During live session, browser calls POST /api/generate-response-stream (SSE).
4. API acquires interview capacity slot (max 12 concurrent).
5. API calls Ollama on AI host (172.31.46.208:11434).
6. Tokens stream back to browser via Server-Sent Events.
7. Voice input: browser posts audio to /api/transcribe-audio.
8. API forwards to Whisper sidecar (172.31.46.208:5001).
9. Transcript and chat history saved to RDS.

Authentication flow

1. POST /api/signup → user row + verification email (SMTP).
2. GET /api/verify-email?token= → mark email verified.
3. POST /api/login → bcrypt verify → JWT returned.
4. Protected routes send Authorization: Bearer <JWT>.

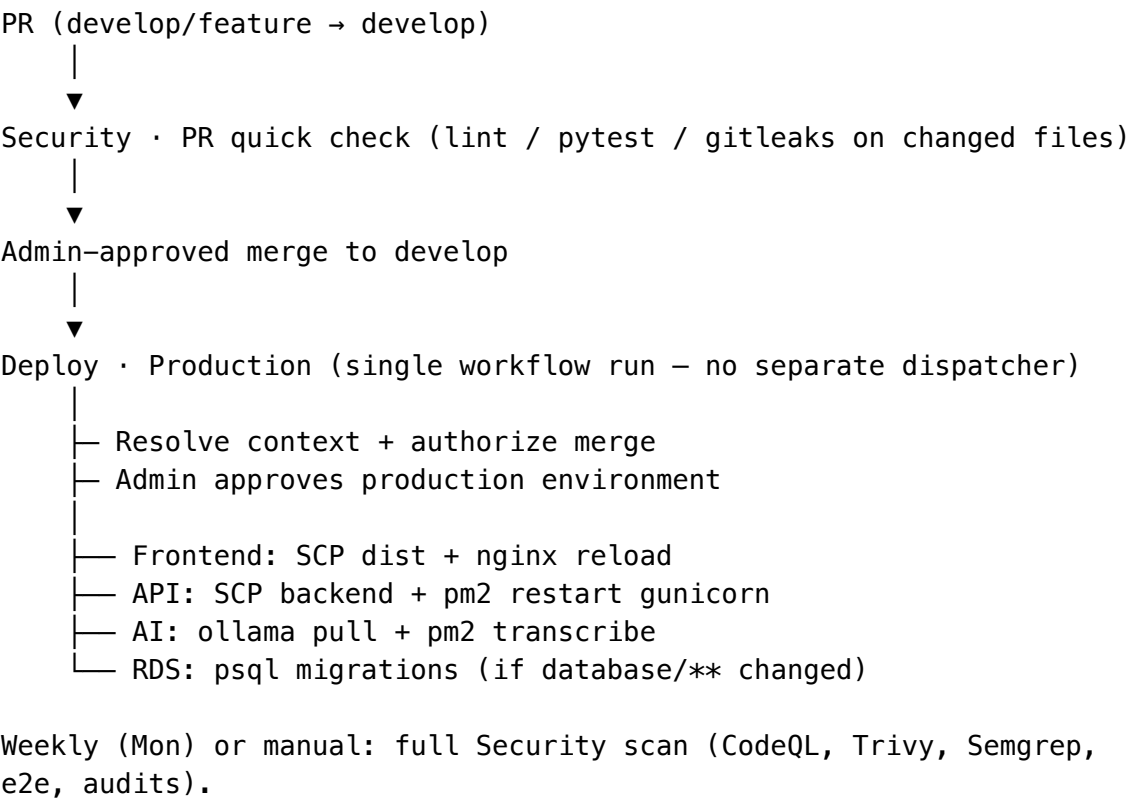
2.7 External integrations

System	Protocol	Purpose
Dodo Payments	HTTPS 443	Checkout, webhooks
SMTP	TCP 587	Password reset, email verification
Anthropic API	HTTPS 443	Optional fallback AI
Mixpanel	HTTPS (frontend)	Product analytics
GitHub Actions	SSH	Deploy to EC2

2.8 Non-functional requirements

NFR	Target
EC2 availability	Mon–Fri 10:00–19:30 IST (scheduled); off Sat–Sun
RDS availability	24/7
Concurrent logged-in users	~100
Concurrent live AI interviews	~10–15
API request timeout	300s (gunicorn)
Interview queue	Max 12 slots, 90s wait
Monthly AWS budget	~\$350–650
Authentication	JWT + bcrypt
File access	JWT-protected /api/files/*

2.9 CI/CD pipeline



Production deploys from **develop only**. Branch **main** is never deployed.

3. Low Level Design (LLD)

3.1 Repository structure

interviewcoach-AI/	
├── frontend/	React SPA (pages, hooks, components)
├── backend/	
│ ├── app.py	Monolithic Flask application + routes
│ ├── common/	Shared modules (auth, db, payments, etc.)
│ ├── INTERVIEW/	InterviewManager + LLM turn logic
│ └── Support-bot/	FAQ support bot
├── database/	schema.sql + SQL migrations
├── scripts/	Deploy, AWS Plan B, dev helpers
└── docs/	Runbooks and architecture

3.2 Frontend

Stack

Item	Detail
React	19
Router	react-router-dom 7
Build	Vite 7
Styling	Tailwind CSS 4
Realtime	socket.io-client (head tracking)
Code editor	Monaco (technical interview questions)

Routes

Path	Page	Auth required
/, /login, /signup	Landing, auth	No
/upload, /dashboard	Resume/JD, dashboard	JWT
/questions	Question configuration	JWT
/interview	Live interview session	JWT
/interview-feedback	Post-interview report	JWT
/admin/logs	Admin log viewer	JWT + allowlist

API communication

- All API calls use /api/* (proxied by nginx to API private IP).
- Auth header: Authorization: Bearer <JWT>.

- Streaming: `POST /api/generate-response-stream` → SSE token events.
- Protected files: `GET /api/files/<path>` (JWT + ownership check).
- WebSocket: `/socket.io` for head-tracking during interview.

3.3 Backend processes

Host	Process	Bind address	Process manager
API	gunicorn (Flask)	0.0.0.0:5000	pm2 backend
AI	ollama	0.0.0.0:11434	systemd ollama
AI	transcribe sidecar	0.0.0.0:5001	pm2 transcribe
Frontend	nginx	:80, :443	systemd

gunicorn (production API):

1 worker × 8 threads, timeout 300s, max-requests 2000

3.4 Configuration

Production: `AWS_SECRETS_MANAGER_SECRET_ID=interviewcoach/prod/app` (JSON payload).

Local development: `backend/.env` + optional SSH DB tunnel overrides.

Variable	Host	Purpose
DB_HOST, DB_PORT	API	RDS connection
JWT_SECRET	API	JWT signing
OLLAMA_HOST	API	<code>http://172.31.46.208:11434</code>
TRANSCRIBE_SERVICE_URL	API	<code>http://172.31.46.208:5001</code>
INTERVIEW_MAX_CONCURRENT	API	Semaphore slots (default 12)
STORAGE_PATH	API	<code>/apps/storage</code>
ENABLE_AI_WARMUP	AI only	true (optional, faster first mic use)

3.5 Backend modules (`backend/common/`)

Module	Responsibility
<code>runtime_config.py</code>	Load AWS Secrets Manager JSON
<code>db.py</code>	PostgreSQL ThreadedConnectionPool
<code>auth.py</code>	JWT issue/verify, bcrypt passwords
<code>storage.py</code>	Resume, audio, avatar file I/O
<code>interview_capacity.py</code>	Bounded semaphore + queue wait
<code>transcribe_remote.py</code>	Forward audio to AI Whisper sidecar
<code>dodo_client.py</code> / <code>dodo_webhook.py</code>	Payment checkout and webhooks
<code>rate_limit.py</code>	In-memory rate limiting

Module	Responsibility
session_store.py	interview_sessions JSONB persistence

3.6 Interview engine (backend/INTERVIEW/)

Component	Role
Interview_manager.py	Stage machine: intro → icebreaker → core → follow-up → Q&A
Interview_functions.py	Ollama prompt calls per stage
unified_turn.py	Single Ollama call per turn
Resumeparser.py	Resume text extraction
interview_config.json	Default interview parameters

Stage flow:

introduction → icebreaker → core_questions → followups → candidate_qna → complete

Capacity guard: interview_turn_slot() acquires semaphore (max 12), waits up to 90s, returns HTTP 503 if busy.

3.7 API endpoints (grouped)

Auth and user

Method	Endpoint	Description
POST	/api/signup	Register new user
POST	/api/login	Login, return JWT
GET	/api/me	Current user profile
GET	/api/verify-email	Email verification

Content

Method	Endpoint	Description
POST	/api/upload-resume	Upload resume file
POST	/api/job-descriptions	Create job description
POST	/api/generate-questions	Generate interview questions
POST	/api/parse-job-description	Parse uploaded JD

Interview

Method	Endpoint	Description
POST/GET	/api/interviews	Create / list interviews
POST	/api/generate-response	Synchronous AI reply
POST	/api/generate-response-stream	SSE streaming AI reply
POST	/api/transcribe-audio	Speech-to-text
POST	/api/transcripts	Save transcript
POST	/api/interview-feedback	Save feedback report

Payments

Method	Endpoint	Description
POST	/api/checkout	Start Dodo checkout
POST	/api/webhooks/dodo	Payment webhook handler

Admin and operations

Method	Endpoint	Description
GET	/api/health	Health check + Ollama status
GET	/api/admin/logs	Log file listing
GET	/api/admin/logs/stream	Live log tail

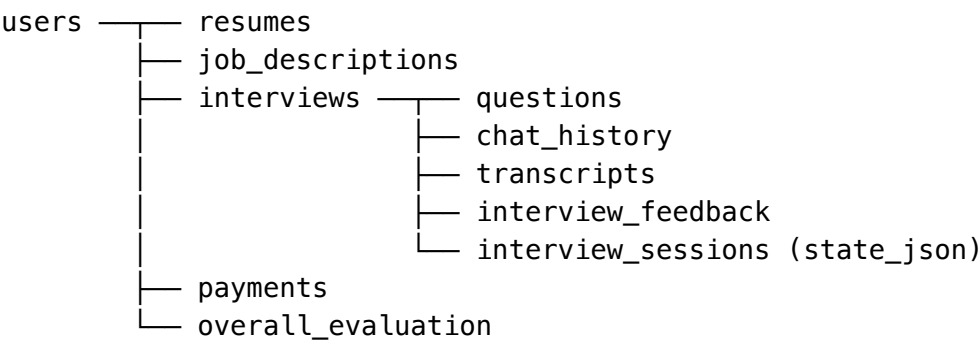
Internal (AI host)

Method	Endpoint	Description
POST	/api/internal/transcribe-audio	Whisper sidecar endpoint

3.8 Database schema

Engine: PostgreSQL · Port **5432** · Extension: pgcrypto

Entity relationships



Core tables

Table	Purpose
users	Accounts (email, password_hash, plan)
resumes	Uploaded CV files
job_descriptions	Job description text
interviews	Interview sessions and status
questions	Generated question bank
chat_history	Turn-by-turn conversation log
transcripts	Final transcript + evaluation JSON
interview_feedback	Post-interview strengths/improvements
interview_sessions	In-progress AI session state (JSONB)
payments	Payment transaction records

RDS endpoint: interviewcoach-db.clmm8cymmic9.ap-south-1.rds.amazonaws.com

3.9 Network design

Port matrix

From	To	Port	Protocol	Usage
Internet	Frontend	443	HTTPS	All user traffic
Internet	Frontend	80	HTTP	Redirect to HTTPS
Frontend nginx	API	5000	HTTP	/api/, /socket.io/, /logs/
API	RDS	5432	TCP	PostgreSQL
API	AI Ollama	11434	HTTP	LLM inference
API	AI Whisper	5001	HTTP	Internal transcription
API	SMTP	587	TCP	Outbound email
API	Dodo / Anthropic	443	HTTPS	Payments / optional AI

nginx proxy rules (Frontend host)

Location	Upstream
/api/	http://172.31.36.78:5000/api/
/functions/v1/	http://172.31.36.78:5000/functions/v1/

Location	Upstream
/socket.io/	http://172.31.36.78:5000/socket.io/ (WebSocket upgrade)
/logs/	http://172.31.36.78:5000/logs/
Static assets	/apps/frontend/current/dist

Security groups

Security group	ID	Inbound rules
Frontend	sg-02d77877092e85c35	TCP 80, 443 from 0.0.0.0/0
AI / API	sg-0f83e275411e1a397	TCP 5000 from Frontend SG; TCP 11434 from API IP /32; TCP 5001 VPC-internal
RDS	(attached to RDS)	TCP 5432 from API SG only

3.10 Infrastructure inventory

Resource	Instance ID	Subnet	Private IP	Public IP
Frontend	i-0d8d448bff2dceb87	subnet-090e9d10afc24205f	172.31.2.39	3.110.2.1
API	i-084ba7dcceefd1636	subnet-00662d4d6964a6ee4	172.31.36.78	15.207.1.1
AI	i-032833ba1cbb49b9b	subnet-00662d4d6964a6ee4	172.31.46.208	13.200.1.1
RDS	interviewcoach-db	RDS subnet group	private endpoint	none

3.11 Server directory layout

Path	Purpose
/apps/frontend/current/	Deployed React static build
/apps/backend/current/	Deployed Flask application
/apps/storage/	User uploads (resumes, audio, avatars)
/apps/logs/	Admin log files

3.12 Deploy sequence

1. Merge to develop triggers **Deploy · Production** (PR quick check already ran on the PR).
2. Admin approves production environment in that same workflow run.
3. **Frontend:** SCP dist/ + nginx config → nginx -t && reload.
4. **API:** SCP backend release → pip install → pm2 restart backend.
5. **AI:** ollama pull llama3.2:3b → restart transcribe sidecar if needed.
6. **RDS:** Apply SQL migrations if database/** changed.

7. Verify: `curl https://ugaanlabs.ai/api/health`.

3.13 Security controls

Control	Implementation
Authentication	JWT (pyjwt), bcrypt password hashes
File access	/api/files/* requires JWT + ownership check
Public storage	/storage/* not exposed in production nginx
Secrets	AWS Secrets Manager; no credentials in git
Admin logs	ADMIN_LOG_VIEWER_EMAILS / USERNAMES allowlist
Internal STT	Optional X-Internal-Token on transcribe sidecar
Rate limiting	In-memory on API (rate_limit.py)
CI security	Bandit, Semgrep, Trivy, CodeQL, Gitleaks

3.14 Observability

Log source	Location	Admin UI key
API stdout/stderr	pm2 logs	server-backend, backend-error
nginx	/var/log/nginx/	server-frontend
Ollama	journalctl	server-ai
Whisper	pm2 transcribe	server-ai
Deploy	/apps/logs/	deployment-live

Health endpoint: GET /api/health → { "status": "healthy", "ollama": { "ready": true } }

3.15 Capacity and tuning

Parameter	Production value	Effect
INTERVIEW_MAX_CONCURRENT	12	Max parallel Ollama turns
INTERVIEW_QUEUE_WAIT_SECONDS	90	Queue timeout before 503
OLLAMA_NUM_PREDICT	384	Max tokens per AI reply
OLLAMA_MODEL	llama3.2:3b	Model loaded on AI host
DB_POOL_MIN / DB_POOL_MAX	5 / 40	PostgreSQL connection pool
gunicorn	1 worker × 8 threads	API concurrency
EC2/RDS schedule	Mon–Fri 10:00–19:30 IST	Cost optimization;

Parameter	Production value	Effect
		weekends off

3.16 Local development

Service	Port	Command
Frontend (Vite)	5173	cd frontend && npm run dev
Backend (Flask)	5001	bash scripts/dev-local.sh
DB tunnel	5433 → RDS 5432	bash scripts/dev-db-tunnel.sh

4. Appendix

4.1 Diagram files (professional AWS icons)

File	Format	Description
docs/diagrams/interviewcoach-aws-production.drawio	draw.io	Recommended — 3 tabs with AWS official icons
docs/diagrams/interviewcoach-architecture.drawio	draw.io	AWS VPC deployment (basic)
docs/diagrams/interviewcoach-logical.drawio	draw.io	Logical layer diagram
docs/ARCHITECTURE.docx	Word	Export of this document
docs/ARCHITECTURE.pdf	PDF	Export of this document
docs/ARCHITECTURE.html	HTML	Print-ready source used for PDF

interviewcoach-aws-production.drawio contains 3 pages:

Tab	Contents
1. AWS Production (Icons)	EC2, RDS, Secrets Manager, EBS, Security Groups, IGW, VPC, Lambda, EventBridge — with ports and IPs
2. Page-to-Page Flow	React routes (Landing → Login → Interview → Feedback) linked to backend services
3. Request Data Flow	Live AI turn: Browser → nginx → API → RDS / Ollama / Whisper

How to open (draw.io / diagrams.net):

1. Go to <https://app.diagrams.net> → **Open Existing Diagram**
2. Select `docs/diagrams/interviewcoach-aws-production.drawio`
3. If icons look like boxes: **More Shapes** (bottom left) → enable **AWS19** or **AWS 2024**
4. Switch tabs at the bottom: *AWS Production · Page-to-Page Flow · Request Data Flow*
5. Export: **File** → **Export as** → **PNG / PDF / SVG** for slides or Confluence

Icon legend (AWS Architecture Style):

Icon	Service	Used for
Orange server	EC2	Frontend, API, AI hosts
Purple cylinder	RDS	PostgreSQL database
Green key	Secrets Manager	interviewcoach/prod/app config
Green disk	EBS Volume	/apps/storage (files — S3 not used today)
Red shield	Security Group	Firewall rules per tier
Orange arch	Internet Gateway	Public internet entry
Green VPC box	VPC / Subnet	Network boundaries
Orange λ	Lambda	Weekday EC2/RDS schedule
Pink bus	EventBridge	Mon–Fri 10:00 / 19:30 IST + weekend force-stop

Raster exports (*.png, *.pdf, *.jpg in this folder) may lag the .drawio source. After editing a diagram, re-export from diagrams.net: **File** → **Export as** → **PNG / PDF**.

4.2 Regenerate Word/PDF exports

From the repository root (requires [pandoc](#)):

```
# Word
pandoc docs/ARCHITECTURE.md -o docs/ARCHITECTURE.docx --from markdown
--toc

# HTML (intermediate)
pandoc docs/ARCHITECTURE.md -o docs/ARCHITECTURE.html --standalone --
toc \
--metadata title="InterviewCoach AI Architecture"

# PDF (via headless Chrome on macOS)
"/Applications/Google Chrome.app/Contents/MacOS/Google Chrome" \
--headless --disable-gpu --print-to-pdf=docs/ARCHITECTURE.pdf \
"file://$(pwd)/docs/ARCHITECTURE.html"
```

4.3 Source references

File	Content
scripts/aws-plan-b/config.env	EC2 IDs, IPs, subnets, security groups
scripts/aws-plan-b/outputs.env	Ollama and transcribe URLs
backend/.env.example	Environment variable reference
database/schema.sql	Database schema
docs/DEVOPS_HANDOFF.md	Operations handoff

File	Content
docs/PRODUCTION_CHECKLIST.md	Production verification checklist

Document maintained by the InterviewCoach engineering team.